PROYECTO INTEGRADOR · DEVOPS

Incident Hub

Informe de pruebas y evidencias

RESULTADO	ALCANCE	ENTREGA
Pruebas automatizadas: 6/6 aprobadas Validaciones complementarias: OK	Interfaz, API, Docker, CI, runner Windows, revisión y release	Repositorio único en GitHub Release v1.0.0 y ZIP de CI

Repositorio: github.com/fernandoperezp3/incident-hub

Evidencias capturadas de la aplicación local y de GitHub. Este reporte es un documento de entrega independiente del código fuente.

Guatemala · 24 de octubre de 2026

Resumen de verificación

Se recorrió el flujo de atención de incidentes en la aplicación, se validó la API con la suite automatizada dentro de Docker y se revisaron las ejecuciones de GitHub Actions y la release publicada.

Área	Verificación realizada	Resultado
Interfaz	Panel, registro, consulta, asignación, estado, bitácora, búsqueda y filtros	**Aprobado**
API	Seis pruebas de salud, CRUD, filtros, paginación y validación	**6/6**
Docker	Suite ejecutada dentro del contenedor de pruebas	**Aprobado**
Calidad	Compilación Python, sintaxis JavaScript y diff limpio	**Aprobado**
CI	Compilación, API, auditoría de dependencias y ZIP de artefacto	**Éxito**
Runner	Runner Windows registrado y workflow self-hosted ejecutado	**Éxito**
Gobernanza	PR aprobado por segunda cuenta y merge a main	**Evidencia**
Release	Versión v1.0.0 publicada con paquete descargable	**Publicado**

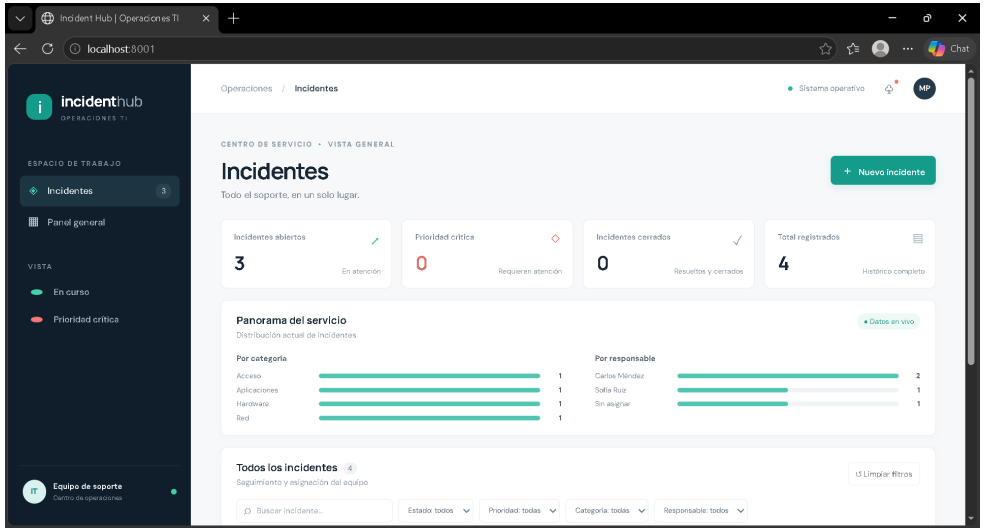
Alcance de la solución

La aplicación implementa un portal de soporte TI con persistencia SQLite, API HTTP y una interfaz web. Permite crear y consultar incidentes, asignar técnicos, registrar actividad, actualizar estados, buscar y filtrar, y visualizar indicadores. El proyecto utiliza un solo repositorio GitHub; el plan se gestiona mediante Issues.

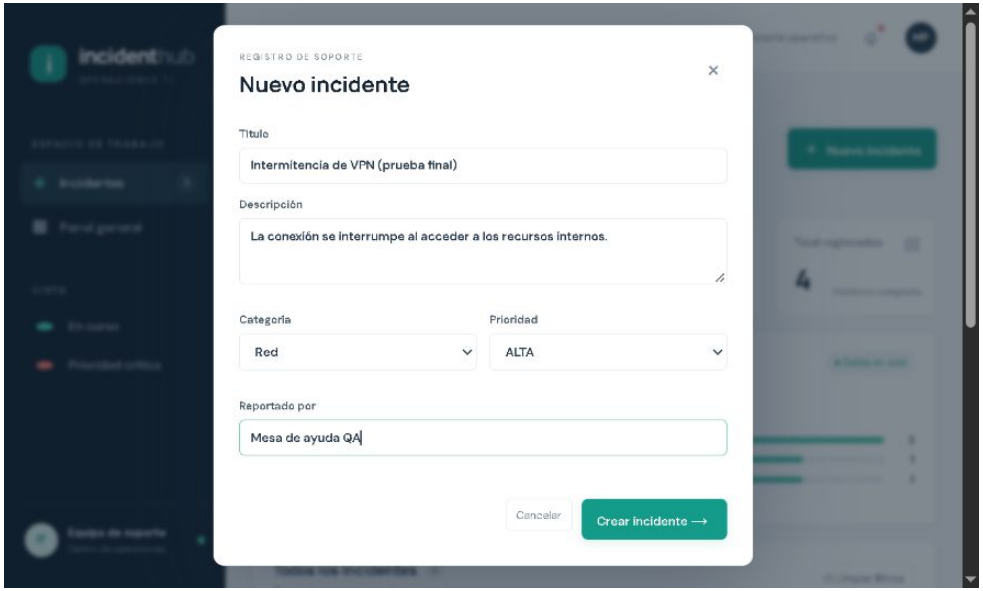
Ambiente de evidencia: ejecución local en Windows y Docker, con interfaz abierta en localhost:8001. La aplicación es una demostración académica local y no incluye autenticación.

Pruebas de interfaz · panel y registro

Comprobación visual del panel operativo y apertura del formulario de alta.



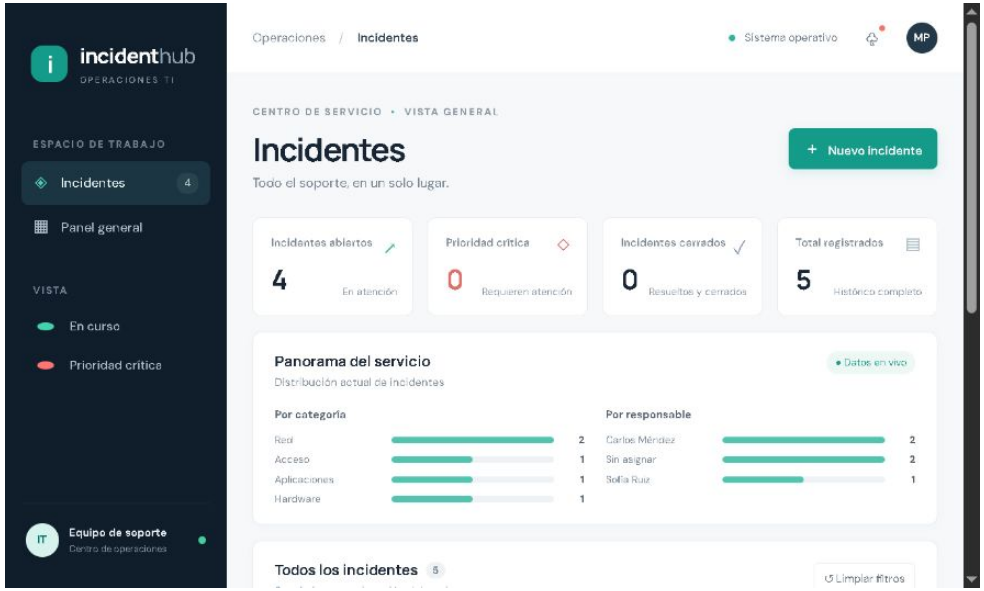
Panel de incidentes en Edge: indicadores, panorama por categoría y responsables.



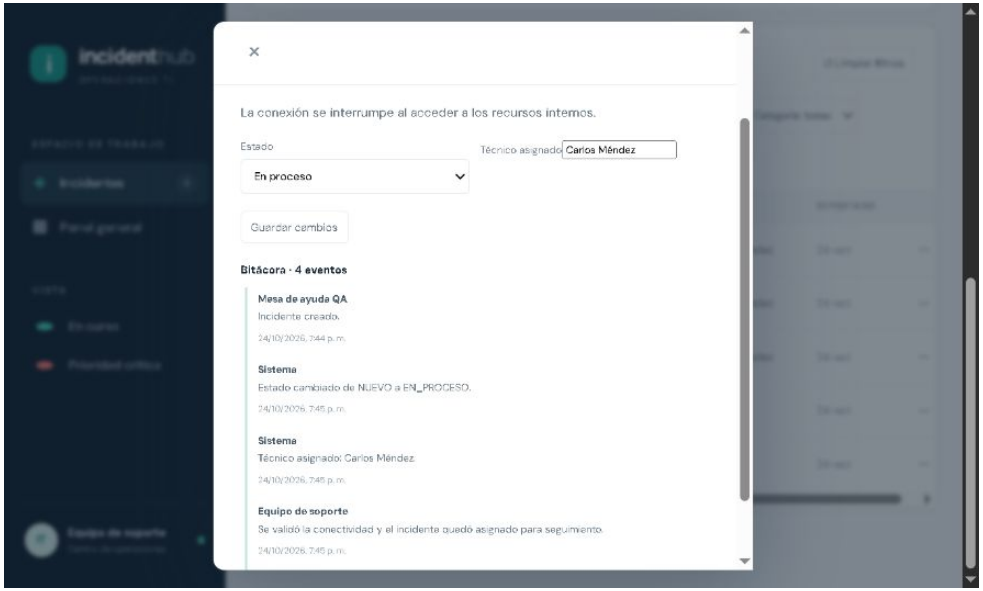
Formulario con título, descripción, categoría Red, prioridad Alta y reportante QA.

Pruebas de interfaz · ciclo del incidente

Se creó INC-0005, se asignó a Carlos Méndez, se cambió a En proceso y se verificó la bitácora de actividad.



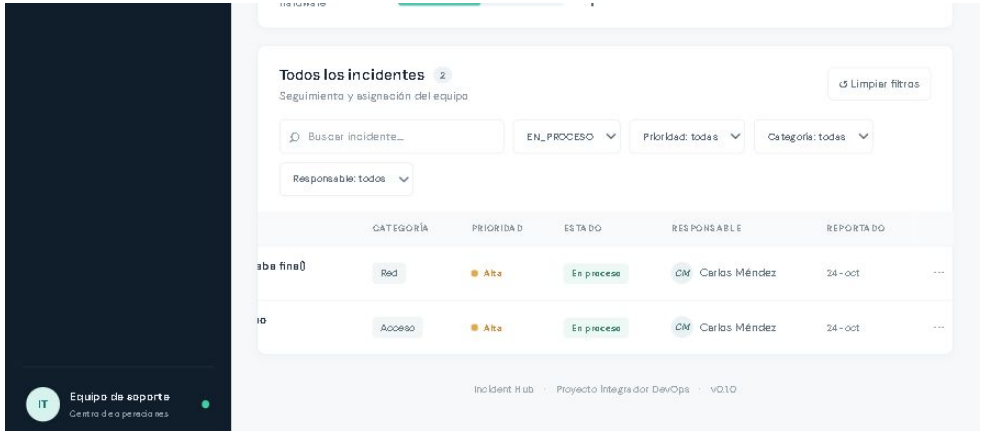
El incidente nuevo aparece en el listado y actualiza los indicadores.



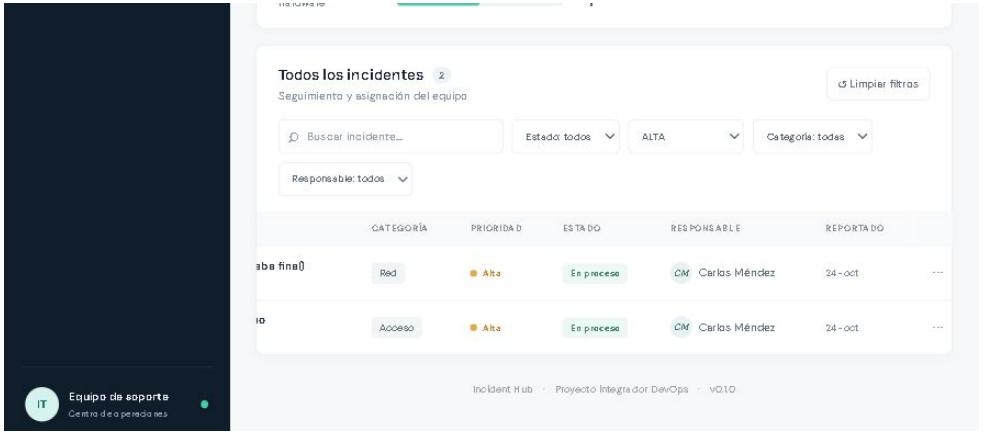
Detalle con estado En proceso, técnico asignado y cuatro eventos de bitácora.

Pruebas de interfaz · estado y prioridad

Los controles reducen el listado según el estado y la prioridad seleccionados. Los resultados visibles corresponden a los filtros aplicados.



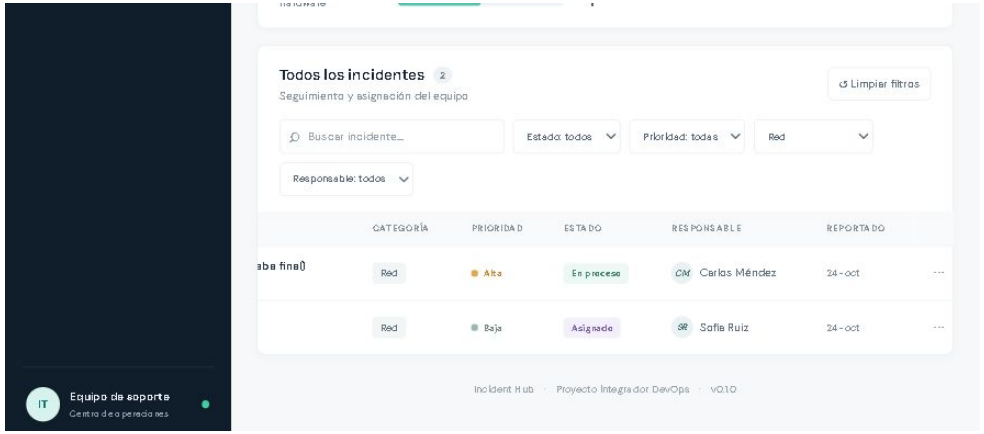
Filtro de estado: EN_PROCESO.



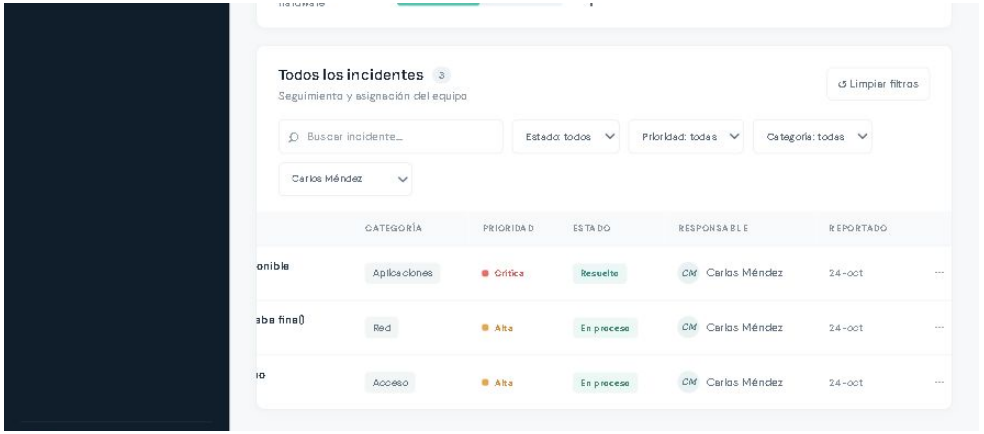
Filtro de prioridad: ALTA.

Pruebas de interfaz · categoría y responsable

Se verificaron las opciones de categoría y responsable y que las filas devueltas correspondieran al criterio elegido.



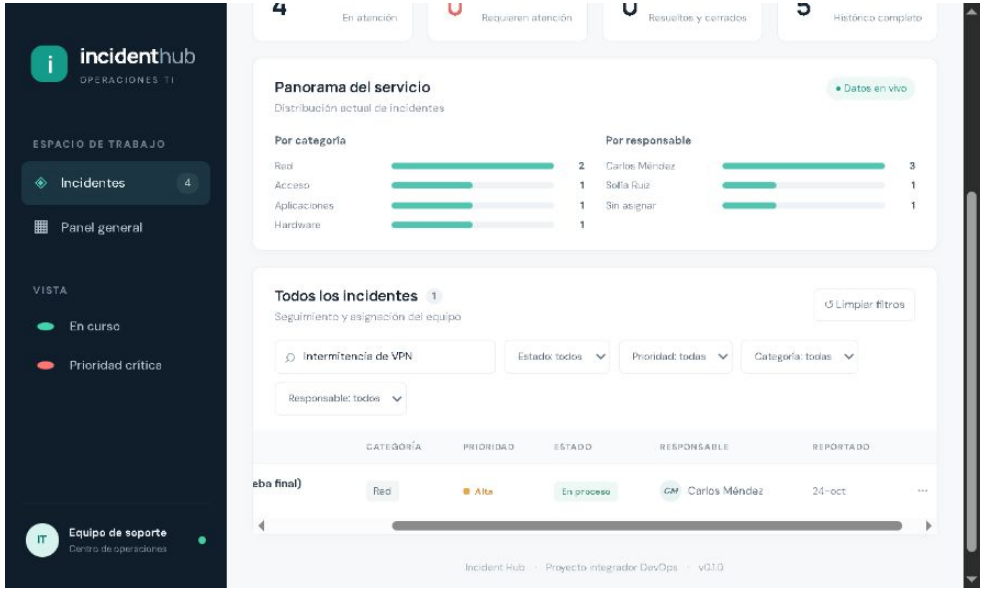
Filtro de categoría: Red.



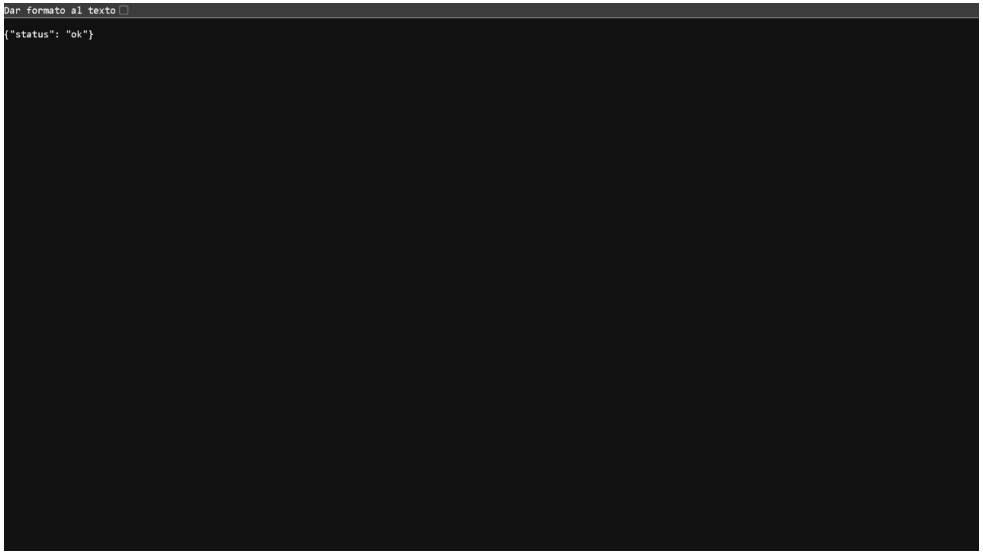
Filtro de técnico: Carlos Méndez.

Pruebas de interfaz · búsqueda y salud de API

La búsqueda por texto devuelve el incidente correspondiente. La ruta de salud confirma que la API responde con estado operativo.



Búsqueda por “Intermitencia de VPN”; se muestra el incidente INC-0005.



Respuesta HTTP de /api/health: estado ok.

Pruebas automatizadas de API

La suite se ejecutó con unittest dentro del contenedor Docker. Resultado: 6 pruebas aprobadas, 0 fallidas.

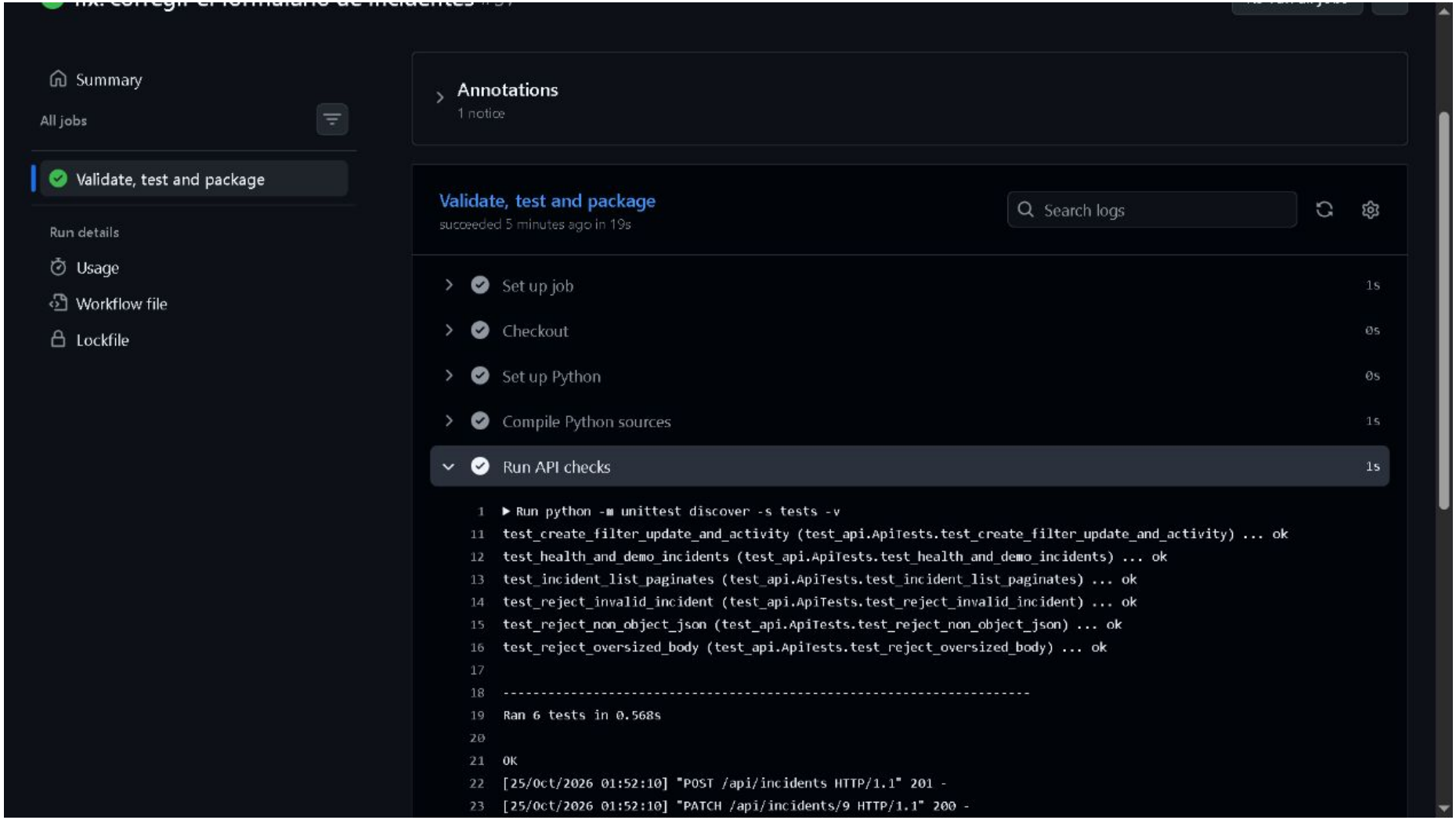
Prueba	Cobertura	Resultado
test_health_and_demo_incidents	Health HTTP 200 y listado inicial de cuatro incidentes.	OK
test_create_filter_update_and_activity	Creación, asignación, actualización, bitácora y filtro por prioridad.	OK
test_incident_list_paginates	Paginación con límite y desplazamiento; límite inválido devuelve 400.	OK
test_reject_invalid_incident	Rechaza datos incompletos con HTTP 400.	OK
test_reject_non_object_json	Rechaza JSON cuyo cuerpo no es un objeto con HTTP 400.	OK
test_reject_oversized_body	Rechaza contenido mayor al máximo con HTTP 413.	OK

Comprobaciones adicionales

python -m compileall -q backend tests · node --check frontend/app.js · git diff --check: completados sin errores. La ejecución de CI también realiza pip-audit sobre requirements.txt y empaqueta el código como ZIP.

Evidencia de ejecución de los seis tests

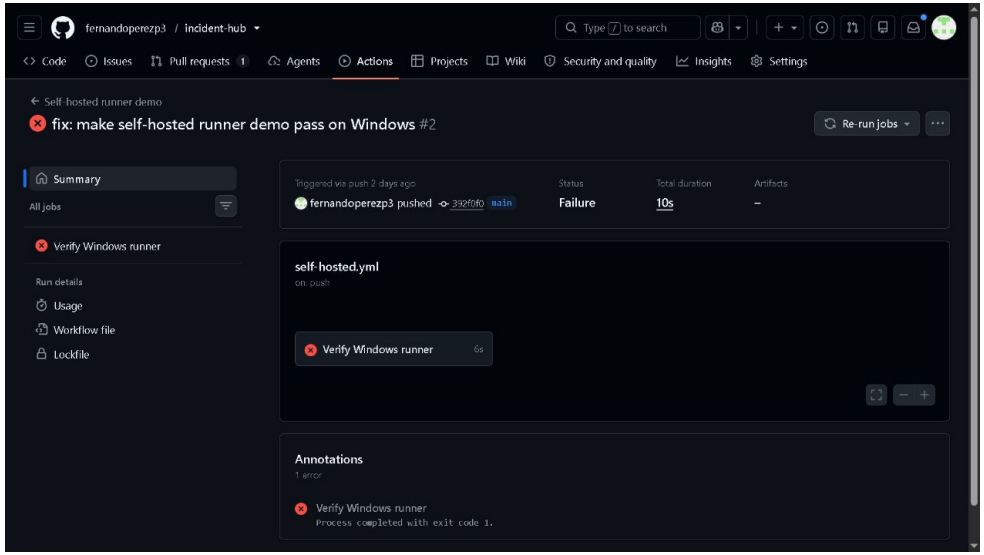
Log de GitHub Actions del job de validación: cada prueba aparece como OK y el proceso finaliza con “Ran 6 tests” y “OK”.



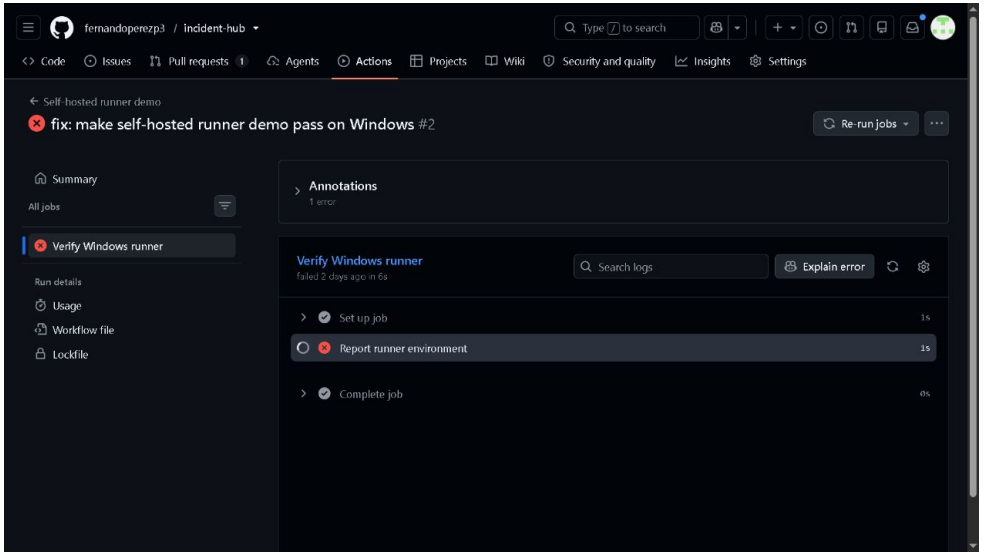
Ejecución CI #37 asociada al PR #47. El job completó satisfactoriamente.

CI · fallo inicial y diagnóstico

La primera ejecución del workflow del runner Windows terminó con exit code 1. Se conserva como evidencia del ciclo de detección y corrección.



Ejecución self-hosted #2: fallo del job Verify Windows runner.

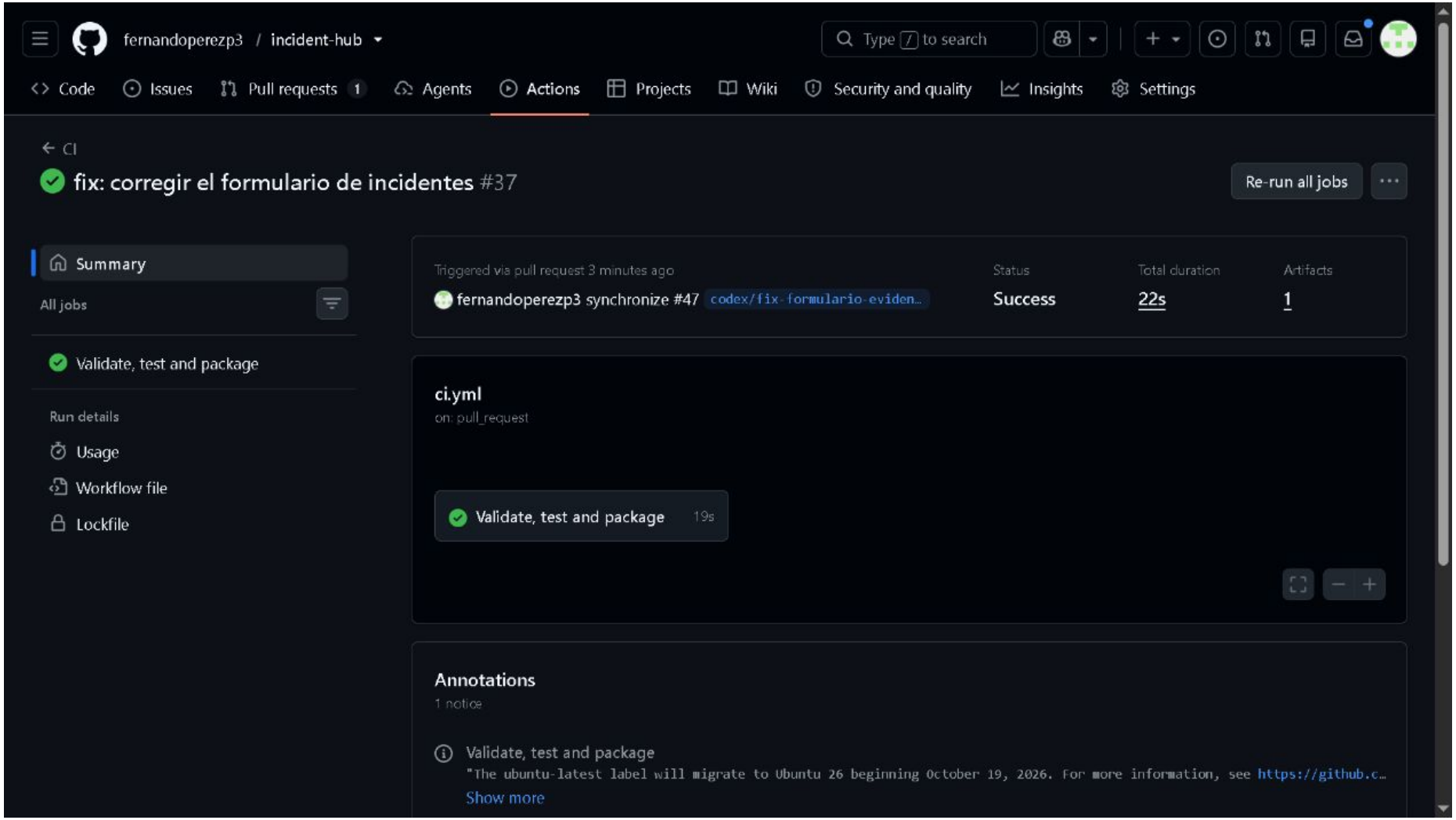


Detalle del job: error y proceso finalizado con código 1.

Tras ajustar el comando del workflow para Windows, la ejecución siguiente del runner finalizó correctamente; ver la evidencia de la página siguiente.

CI · ejecución exitosa y artefacto

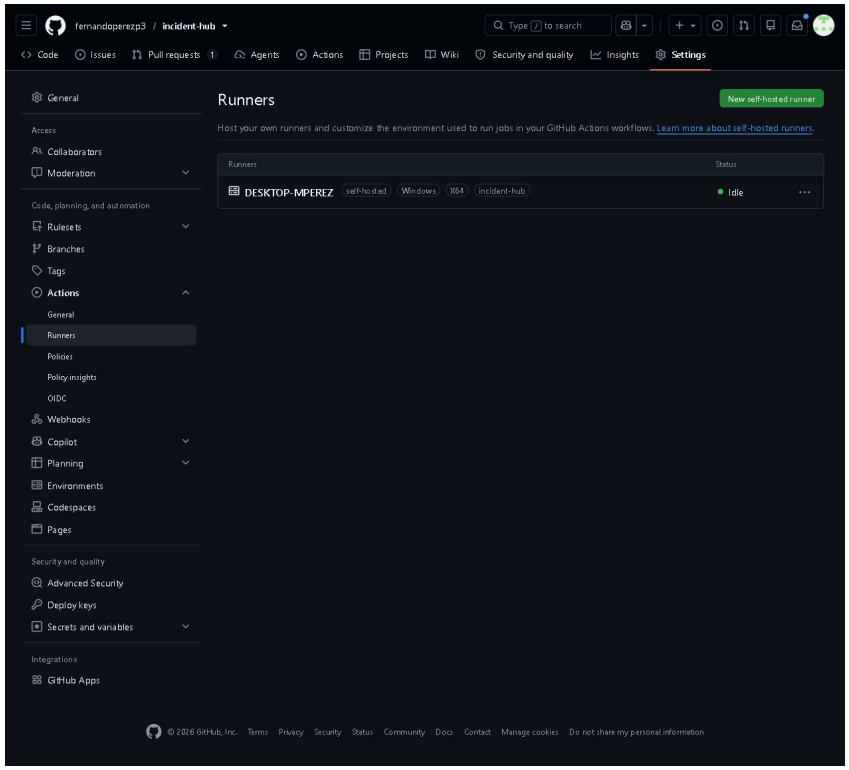
La validación de CI pasó en la ejecución #37 del PR #47. El job ejecutó compilación, suite API, auditoría de dependencias y empaquetado; GitHub muestra un artefacto generado.



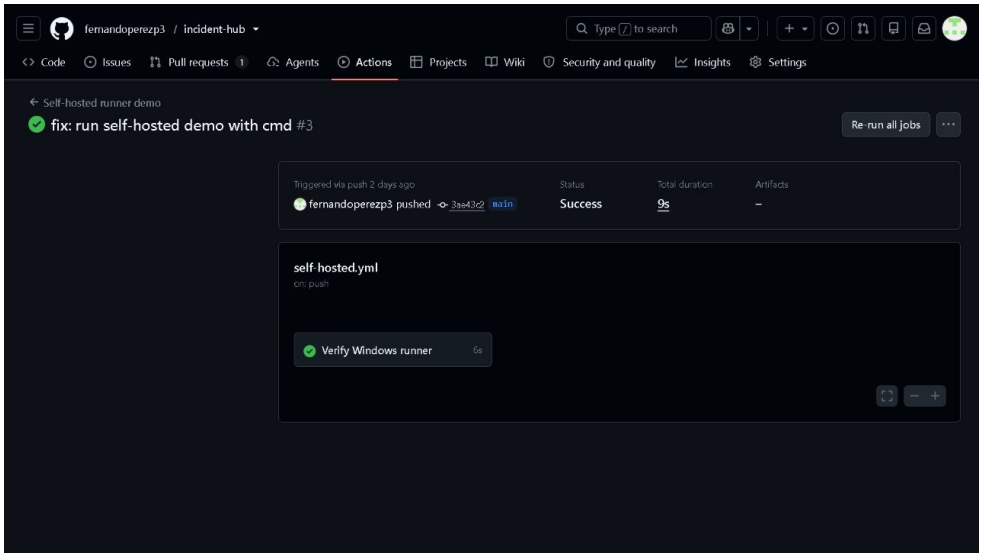
Estado: Success · duración mostrada: 22 s · 1 artefacto.

Runner Windows · registro y ejecución

El runner self-hosted DESKTOP-MPEREZ figura registrado e inactivo cuando no tiene trabajos. El workflow de demostración se ejecutó correctamente sobre las etiquetas configuradas.



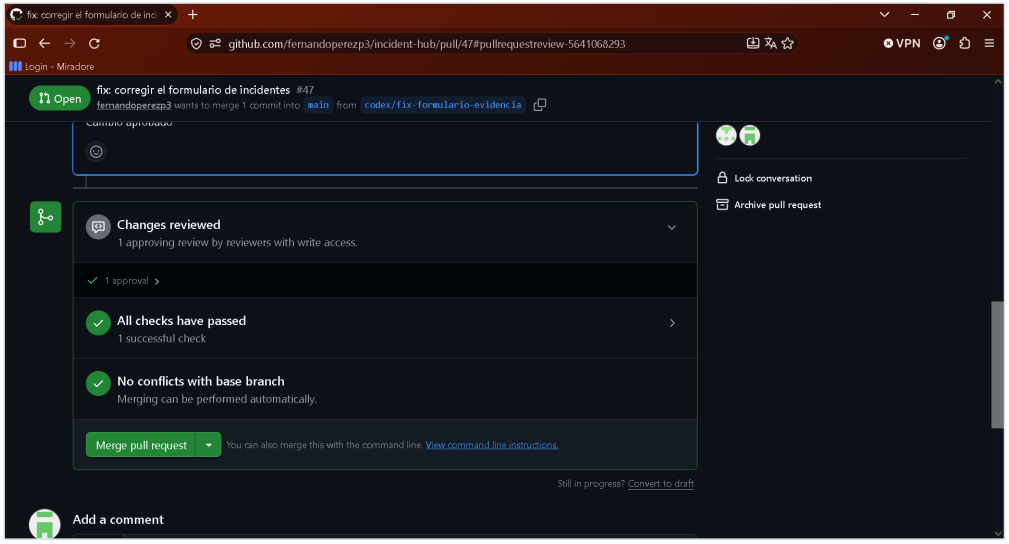
Runner registrado en GitHub con etiquetas self-hosted, Windows, X64 e incident-hub.



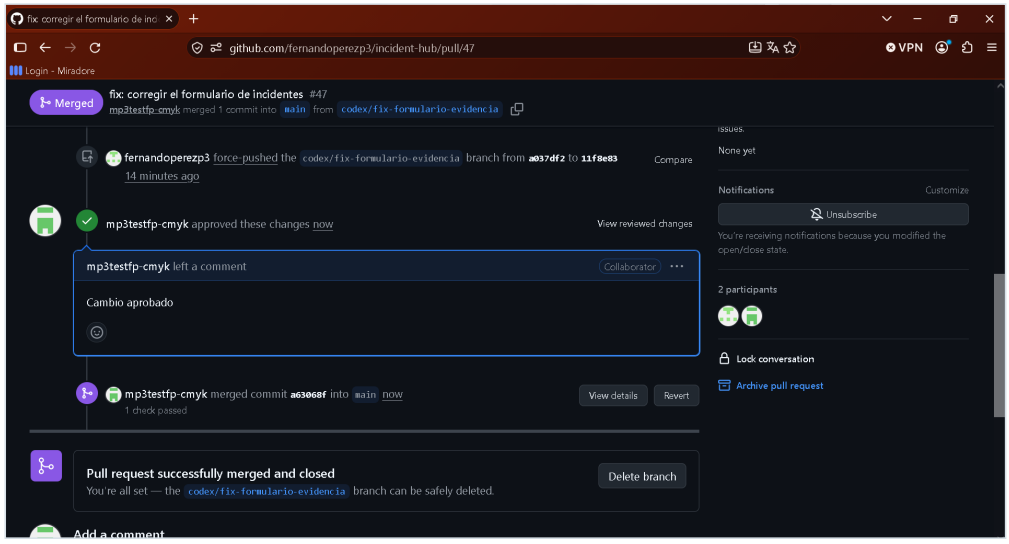
Ejecución self-hosted #3: Verify Windows runner completó en 9 s.

Aprobación y merge del PR #47

El PR de corrección del formulario fue revisado por **mp3testfp-cmyk**. GitHub muestra una aprobación, todos los checks aprobados y el merge a **main**.



Checks aprobados y una aprobación registrada.

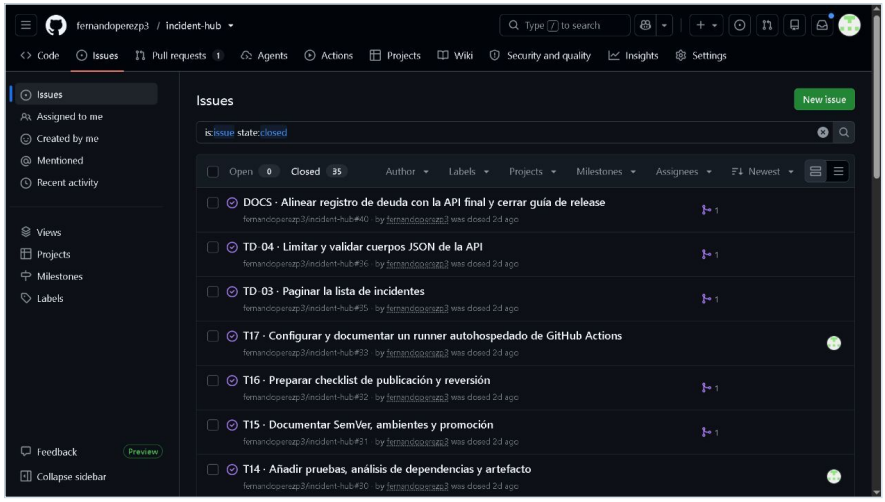


mp3testfp-cmyk aprobó los cambios y GitHub confirmó el merge.

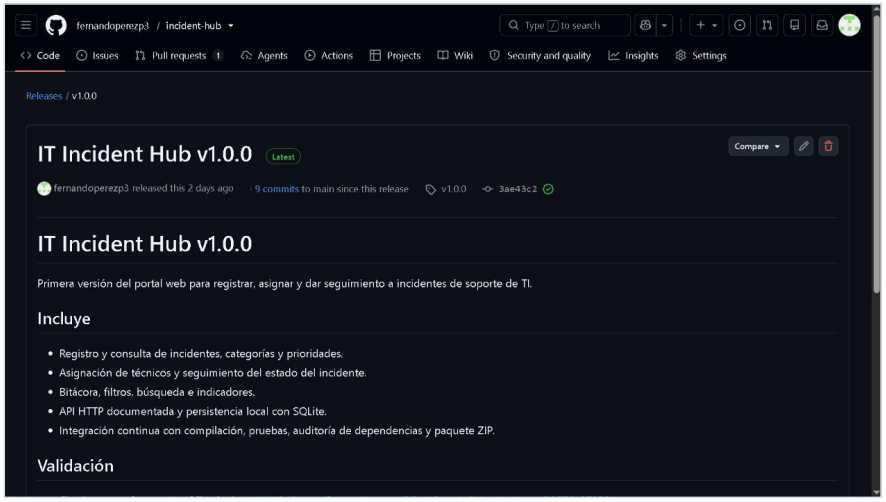
PR #47 aprobado y fusionado a main

Plan de trabajo y versión publicada

El repositorio mantiene el backlog en GitHub Issues. La versión **v1.0.0** fue publicada en GitHub Releases; CI produce el paquete ZIP como artefacto.



Backlog del proyecto en Issues de GitHub.



Release v1.0.0 con descripción y validaciones.

Cierre

La aplicación se verificó en entorno local, los seis tests automatizados pasaron y CI y el runner Windows cuentan con ejecuciones de fallo inicial y éxito posterior. El PR #47 quedó aprobado y fusionado a main.

La solución es una demostración académica local, sin autenticación y no publicada como servicio abierto en Internet.

Revisión final frente al entregable

Repositorio público: **fernandoperezp3/incident-hub** | Rama **main** | Commit revisado **a63068f** (PR #47). La instrucción más reciente del ingeniero establece un repositorio y GitHub; por ello, Issues y Actions reemplazan Azure Boards/Pipelines en la evidencia.

Matriz de requisitos

Área	Evidencia verificada	Resultado y observación
Aplicación	Frontend web, API REST, SQLite; incidentes con asignación, estados, actividad, filtros y métricas.	Implementado. Se ejecutaron seis pruebas de API; el despliegue es local y usa datos ficticios.
Backlog	Dos épicas, cuatro features, ocho historias y tareas T01–T17 en docs/backlog.md, vinculadas a GitHub Issues.	La estructura está documentada. El Markdown conserva frases de estado antiguas: T17 figura abierto aunque el Issue #33 está cerrado; #40 también aparece como en curso aunque está cerrado.
Git y revisiones	12 PRs fusionados identificados. PR #47 incluye aprobación independiente, checks aprobados y merge.	Se verifican aprobaciones independientes en #45, #46 y #47. Si la rúbrica requiere seis PRs con aprobación secundaria, faltan evidencias para completar ese umbral.
CI y seguridad	GitHub Actions valida Python, pruebas, pip-audit y artefacto; existen ejecuciones iniciales fallidas y correcciones posteriores.	Se conserva evidencia de CI y del runner Windows self-hosted exitoso. docs/security.md todavía marca TD-05 como pendiente, aunque Issue #33 y el run del runner acreditan su ejecución.
Versiones y publicación	Release v1.0.0 publicada en GitHub con ZIP y guía de reversión.	El ZIP de v1.0.0 corresponde a un commit anterior al arreglo del formulario en PR #47. El main actual tiene el arreglo; para un ZIP equivalente se requiere publicar una versión patch posterior.
Demostración	Capturas de interfaz, formulario, flujo de revisión/aprobación y merge integradas en este informe.	La aplicación no tiene URL pública ni autenticación. Se ejecuta en local; no debe exponerse a Internet ni recibir datos reales.

Conclusión: el código y las pruebas locales están listos para compartir mediante el enlace del repositorio. Para una entrega formal, conviene adjuntar este PDF y aclarar que el ZIP v1.0.0 es previo al PR #47; si el ingeniero requiere paquete actualizado o seis aprobaciones independientes, esas son las dos brechas concretas.

Ejecución final en PowerShell y Docker

Verificación realizada en Windows 10 con PowerShell 7.6.5, Python 3.12.14 y Node.js. El checkout local estaba en main, sin cambios pendientes, y coincidía con el merge del PR #47.

Resultados

Comando o validación	Resultado	Detalle
python -m compileall -q backend tests	**Correcto**	Compilación de módulos Python sin errores.
python -m unittest discover -s tests -v	**6/6 aprobadas**	Flujos de API, filtros, actividad, paginación y validaciones de carga.
node --check frontend/app.js	**Correcto**	Sintaxis JavaScript válida.
git diff --check	**Correcto**	Sin errores de whitespace en el árbol revisado.
docker compose config --quiet	**Correcto**	Compose se pudo validar. Docker emitió una advertencia de acceso a preferencias del usuario.
docker compose --profile test run --rm tests	**6/6 aprobadas**	Las pruebas también pasaron en el servicio de tests dentro de Docker.
GitHub Actions / PR #47	**Correcto**	Check exitoso y ZIP generado; SHA-256: 57e130543f178783f815595042b30a3714f46f5d09364eea19a0e0c10267bdc4.
Runner Windows self-hosted	**Correcto**	La ejecución exitosa está enlazada desde la documentación del repositorio.

Limpieza realizada

Se retiraron archivos temporales y directorios __pycache__ generados durante la revisión. Las capturas permanecen solo en el PDF y fuera del repositorio. Se conservó database/incidents.db, que contiene datos de demostración locales del usuario y está excluida de Git.

Archivos de entrega

Repositorio: https://github.com/fernandoperezp3/incident-hub
Registro detallado de esta verificación: Verificacion_Final_PowerShell.md